

CMU 11-868

目标: HW1: 看代码结构 → 自己实现一个最简单的kernel → 再理解 tiled Matmul

↓
看 6 week 的 [在GPU上加速的 transformer x2] + HW4 建议看/做关键部分

Distributed Model Training I
Distributed Model Training II
Distributed Model Training III
Mixture-of-Expert
Memory Optimization in Distributed Training.

+ $\frac{HWS}{DP + PP}$
Most important

↓ Quantization + flashattention

Model Quantization I
Model Quantization II / GPTQ
Optimizing Attention for Modern Hardware, 由 Tri Dao 讲 FlashAttention.

↓
LLM Serving with SGL
Efficient LLM Inference with PagedAttention and vLLM
Serving with Disaggregated Prefill-Decoding
Better KV Cache for LLM Serving. LLM系统

prefill + decode
KV Cache
Continuous Batching
Paged Attention

最有价值

↓
HW6 做 DeepSpeed zero + LORA + SGLang inference / Radix Attention / cache aware scheduling

HW6 至少完整读代码, 并且能跑的部分跑起来

GPU

Accelerating Transformer on GPU I / II

Distributed

Distributed Model Training I / II / III

Mixture of Expert

Memory Optimization / ZeRO

Inference

Quantization I / II

FlashAttention

Serving

SGLang

vLLM / PagedAttention

P/D Disaggregation

KV Cache Systems

RL Systems

Efficient Reinforcement Learning System for LLMs

+

HW

内容

你的处理

HW1

CUDA kernels

● 做 1-2 个 kernel

HW2

Autograd / MiniTorch

✗ 跳

HW3

Transformer 等

✗ 跳

HW4

GPU Optimization

● 读 + 做关键部分

HW5

Distributed Training

● 重点做

HW6

DeepSpeed / SGLang

● 重点做

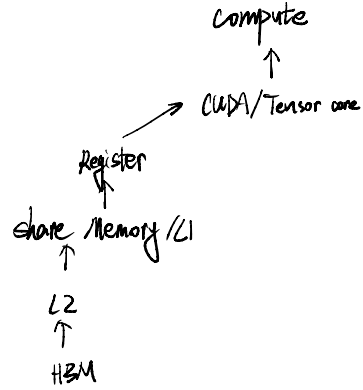
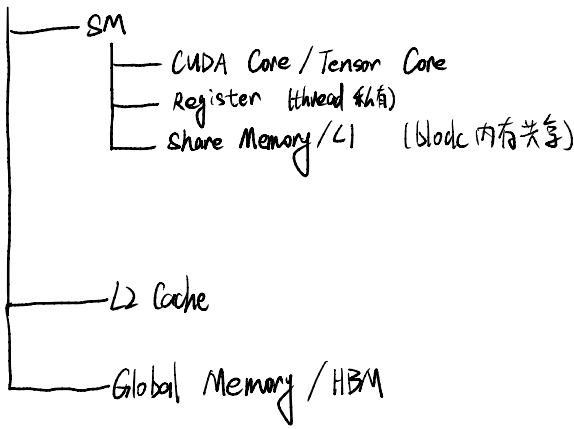
HW7

RLHF Systems

● 重点读架构 + 部分实现

GPU 硬件架构

GPU



Scheduler

Block Manage

Paged Attention

KV Cache

Continuous Batching

Prefix Cache

Preemption

Chunked Prefill



SGLang / Radix Attention

P/D Disaggregation

KV Cache Systems

跨请求 KV reuse

多实例 / 多 GPU Serving

Serving architecture

Rollout engine + training engine

vLLM / SGLang + FSDP / Megatron

parameter sync

data transfer

resource allocation

前面过于具体，但我是想学 CS336-2 中
没有的部分

新版 11-868 学习路线

阶段	CMU 内容	怎么处理	时间上限	最终必须拿走什么	 
0	GPU / CUDA / Transformer Acceleration	结束，跳过	0	不再重复	
1	Distributed Training I / II / III	重点差集学习	2 天	TP、PP、组合并行、通信与显存 cost model	
2	MoE + ZeRO / Memory Optimization	重点差集学习	1~1.5 天	EP、MoE communication、ZeRO/FSDP 在整个并行体系的位置	
3	Quantization + FlashAttention	只做查漏	0.5~1 天	有新东西拿新东西，没有就跳	
4	SGLang	重点	1 天	nano-vLLM 之外的 scheduling / prefix reuse / RadixAttention	
5	vLLM	快速扫	0.5 天	只找 nano-vLLM 与真实 vLLM 的差距	
6	P/D Disaggregation + KV Cache Systems	重点	1~1.5 天	从单实例 serving 进入真实 serving architecture	
7	RL Systems	重点	1~1.5 天	Rollout Engine $\leftrightarrow$ Training Engine 的系统组织	
8	HW / Paper	按项目取用	不单独占时间	项目需要什么再看什么	

Model Parallelism

PP: 按 layer 切模型 + 靠 micro-batch 把 pipeline 填起来 + 主要矛盾是 bubble

or

TP: 切 layer. 切完之后每张 GPU 里有什么? 什么时候必须把结果合起来? All Reduce

一个典型集群可能:

```
</> 纯文本
Node 0
GPU0 GPU1 GPU2 GPU3
└── TP Group ──┘
      |
      v PP
Node 1
GPU4 GPU5 GPU6 GPU7
└── TP Group ──┘
```

TP 分为 Column Parallelism
Row Parallelism
↓
CR 配对可以减少通信

为什么这样很自然?

因为 TP 在每一层里面都可能通信, 所以最好放在 NVLink / NVSwitch 这种高速互联范围内。

PP 通信主要发生在 stage 边界, 频率和通信模式不同, 因此可以负责继续跨更大的集群扩展。Node 网络慢

MoE + All-to-All

Expert Parallel: 不同的 expert 放在不同的 GPU 上

All-to-All: GPU 之间会互发 token, 每张 GPU 给不同的 GPU 发送不同数据

缺点: 不同 GPU 处理的数据量不同 (load Imbalance), 有些 expert 会出现掉队现象。

总参数量 ↑ per-token compute 没有同比例 ↑



Routing + All-to-All communication + load balancing

MoE 用稀疏计算换更大的模型容量,
但把一部分计算问题变成了通信和负载均衡的问题

ZeRO / FSDP

FSDP 主要是切 model state 的存储. 有缩时分片, 计算时临时聚合

大模型会用 TP + FSDP

↓ 某一层 model state 太大
解决问题: 单个 layer 太大

FSDP 的通信量大: Forward 的 parameter All Gather + Backward 的 parameter All Gather + Gradient Reduce Scatter

Prefetch: 提前把下一层需要的参数 All Gather 起来, 让通信和计算可以重叠起来

Overlap: 让 communication 和 computation 同时进行

FSDP 在 Memory 和 communication Overlap 之间找平衡

Prefetch 太少

→ GPU 等通信

Prefetch 太多

→ 显存占用上升

SGlang / Radix Attention

Radix Attention 进一步解决 "不同请求之间, 已经算过的 KV 如何复用" → Radix tree 本质上是压缩 trie

↓
调度优化

Paged Attention: 问题: KV cache 物理显存如何高效分配
解决: Block / Page 避免 fragmentation / over-allocation

Paged Attention 更偏 physical KV memory management.

Radix Attention 更偏 logical KV reuse across requests.

Radix Attention: 问题: 多个请求之间已经算过的 KV, 能不能找到复用?

解决: Radix Tree 组织共享 prefix 做 longest-prefix matching

P/D Disaggregation

Prefill 和 Decode 的硬件需求不一样, 把它们塞在同一批 GPU 上会互相干扰; P/D Disaggregation 就是把它们分开。

↓
计算受限 显存受限

Prefill → TTFT (Time To First Token)

Decode → TPOT (Time Per Output Token)

P/D 分离使得 P/D 不互相干扰 分别优化 TTFT TPOT, 但代价是 KV Cache 从 prefill 中到 Decode 需要通信

P/D 分离产生的两个问题: Placement 和 KV Communication

Placement | 部署/放置策略:

需要考虑: 计算资源 + 显存/带宽 + GPU 互联拓扑 + KV Cache 传输距离

进一步演变成: 整个 GPU 集群怎么部署 serving 实例

KV Cache communication:

Prefill 的 KV 如何更快地送给 decode

KV Cache system

重新计算

vs

传输已有结果

→ Cache Gen: 将 KV Cache 压缩之后再传 Cache Gen makes bits, not tensors

争取做到: 压缩后传输时间 + 解压时间 < 原始 KV 传输时间

CacheBlend

解决: prefix reuse 容易; non-prefix reuse 困难

Prefix Cache
= exact reuse (精确复用)

CacheBlend
= approximate reuse + selective recomputation
(近似复用 + 选择性重计算)

另外: 跨 chunk 复用不仅有 cross-attention 问题, 还有 position (位置) 变化问题

Rollout Engine & training Engine

Rollout Engine: 用当前 policy model (策略模型) 对 prompt 做大量生成, 得到训练数据

Rollout Engine 关心的是:

</> 纯文本

一秒钟能生成多少 token?

GPU serving throughput (服务吞吐) 多高?

KV Cache 怎么管理?

不同长度 response 怎么调度?

粗暴地理解为 vLLM 的世界

Training Engine 更关心:

FSDP / ZeRO

TP / PP

Megatron

Gradient communication

Optimizer states

Activation memory

粗暴地理解为 FSP/Megatron 世界

RL system 不只是“训练系统+推理系统放一起”, 而是同一模型需要不断在训练形态和推理形态之间切换

两种系统设计:

Colocated (共置)

同一批 GPU

Rollout

↓

切换

Training

↓

切换

Rollout

Disaggregated (分离式)

GPU Pool A

Rollout Engine

↕ Weight Sync

GPU Pool B

Training Engine

On policy VS off policy

Synchronization (同步)

数据新鲜
但等待多

Asynchrony (异步)

GPU 利用率高
但 policy staleness 增加

Rollout Engine (采样引擎) 本质上是一个高吞吐 LLM inference system. 负责产生 response / trajectory

Training Engine: 负责 forward, backward, optimizer step, 通常使用 FSDP / Megatron 等训练系统

两者虽然使用同一 policy model, 但最适合 parameter layout / parallelism 不同. 因此会出现 resharding (重新分片) 和 weight sync (权重同步)

想让 Rollout 和 Training overlap (重叠) 可以提高 GPU 利用率, 但会引入 policy staleness (策略陈旧 off-policy)

LLM RL System

Prompt



Rollout Engine
(vLLM / SGLang)



Responses



Reward / Advantage



Training Engine
(FSDP / Megatron)



Policy Update



Weight Sync / Reshard



Rollout Engine

系统优化围绕四件事:

</> 纯文本

- ① Rollout 怎么更快
- ② Training 怎么更快
- ③ 两边 GPU 怎么分
- ④ Policy weights 怎么同步